

JavaScript API Fetch

[Back to JS page](#)

Note: This topic covers the Fetch API from the Web API perspective. For async/await patterns with fetch, see [05.2061-js-AsyncFetch](#).

Table of Contents

- [JavaScript API Fetch](#)
 - [The Response Object](#)
 - [Example 1](#)
 - [The ok Property](#)
 - [Example 2](#)
 - [The status Property](#)
 - [Example 3](#)
 - [The url Property](#)
 - [Example 4](#)
 - [Loading Multiple Files](#)
 - [Example 5](#)
 - [Document](#)
 - [Reference](#)

The Response Object

The `fetch()` method returns a Promise that resolves to a `Response` object. The Response object contains information about the HTTP response:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript API Fetch</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The Response Object</h4>
<p id="demo"></p>

<script>
fetch('https://jsonplaceholder.typicode.com/users/1')
  .then(function(response) {
    let text = "Response Object Properties:<br><br>";
    text += "type: " + response.type + "<br>";
    text += "ok: " + response.ok + "<br>";
    text += "status: " + response.status + "<br>";
    text += "statusText: " + response.statusText + "<br>";
    text += "url: " + response.url + "<br>";
    text += "headers: " + response.headers.get('content-type') + "<br>";
    text += "redirected: " + response.redirected;
    document.getElementById("demo").innerHTML = text;
    return response.json();
  })
  .then(function(data) {
    document.getElementById("demo").innerHTML += "<br><br>User: " + data.name;
  });
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

The ok Property

The `ok` property returns `true` if the HTTP status is in the range 200-299 (success range):

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript API Fetch</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The ok Property</h4>
<p id="demo1"></p>
<p id="demo2"></p>

<script>
// Successful request (200 OK)
fetch('https://jsonplaceholder.typicode.com/users/1')
  .then(function(response) {
    document.getElementById("demo1").innerHTML =
      "Valid URL - status: " + response.status + "<br>" +
      "ok: " + response.ok + "<br>" +
      "Response ok means success!";
  });

// Invalid request (404 Not Found)
fetch('https://jsonplaceholder.typicode.com/users/nonexistent')
  .then(function(response) {
    document.getElementById("demo2").innerHTML =
      "Invalid URL - status: " + response.status + "<br>" +
      "ok: " + response.ok + "<br>" +
      "Response not ok - check response.ok!";
  });
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

The status Property

The `status` property returns the HTTP status code (200, 404, 500, etc.):

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript API Fetch</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The status Property</h4>
<p id="demo"></p>

<script>
function checkStatus(url) {
  return fetch(url)
    .then(function(response) {
      let msg = "URL: " + url + "<br>";
      msg += "Status: " + response.status + " " + response.statusText + "<br>";

      if (response.status === 200) msg += "✓ Success (200 OK)";
      else if (response.status === 201) msg += "✓ Created (201)";
      else if (response.status === 404) msg += "✗ Not Found (404)";
      else if (response.status >= 400 && response.status < 500) msg += "✗ Client Error (" + resp
      else if (response.status >= 500) msg += "✗ Server Error (" + response.status + ")";

      msg += "<br><br>";
      return msg;
    });
}

// Check multiple URLs
Promise.all([
  checkStatus('https://jsonplaceholder.typicode.com/users/1'),
  checkStatus('https://jsonplaceholder.typicode.com/users/nonexistent'),
])
  .then(function(results) {
    document.getElementById("demo").innerHTML = results.join("");
  });
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

The url Property

The `url` property returns the URL of the response. It can be different from the request URL if there were redirects:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript API Fetch</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The url Property</h4>
<p id="demo"></p>

<script>
const requestUrl = 'https://jsonplaceholder.typicode.com/users/1';

fetch(requestUrl)
  .then(function(response) {
    document.getElementById("demo").innerHTML =
      "Request URL: " + requestUrl + "<br>" +
      "Response URL: " + response.url + "<br><br>" +
      "Same URL? " + (requestUrl === response.url);
  });
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

Loading Multiple Files

You can load multiple files using `Promise.all()` with multiple fetch requests:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript API Fetch</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Loading Multiple Files</h4>
<p id="demo"></p>

<script>
// Fetch multiple resources in parallel
const urls = [
  'https://jsonplaceholder.typicode.com/users/1',
  'https://jsonplaceholder.typicode.com/users/2',
  'https://jsonplaceholder.typicode.com/users/3'
];

Promise.all(urls.map(function(url) {
  return fetch(url).then(function(response) {
    if (!response.ok) throw new Error("HTTP " + response.status);
    return response.json();
  });
})))
  .then(function(users) {
    let text = "Loaded " + users.length + " users:<br><br>";
    for (let user of users) {
      text += user.name + " - " + user.email + "<br>";
    }
    document.getElementById("demo").innerHTML = text;
  })
  .catch(function(error) {
    document.getElementById("demo").innerHTML = "Error: " + error.message;
  });
</script>

</body>
</html>

```

Example 5

Result [View Example](#)

Document

Document in project

You can [Download PDF](#) file.

Reference

- [W3Schools JavaScript API Fetch](#)
- See also: [05.2061-js-AsyncFetch](#) for async/await patterns